Government First Grade College for Women, Byrapura-571124
T.Narasipura Taluk, Mysore Dist,Karnataka
Affiliated to Mysore University

Lecture Notes on- **Computer Graphics Lab (L: T: P:: 0:0:4)**

Prepared by: RAKESH. K

Department: Computer Science
Syllabus: III BCA- V Semester- SEP
Software Tool: Dev C++ 5.11-Latest
Install: MinGW
Videolink to setup Graphics:
https://www.youtube.com/watch?v=CHFyEnlMnxg&t=2s

Part -A

1. Program to draw basic graphics primitives
2. Program to implement Digital Differential Analyzer (DDA) Line Drawing Algorithm.
3. Program to implement Bresenham's Line Drawing Algorithm
4. Program to draw a circle using Bresenham's Circle Algorithm
5. Program to perform 2D Transformations
6. Program for Line Clipping using Cohen–Sutherland Algorithm
7. Program for Polygon Clipping
8. Program to perform 3D Transformations

(Note: The above-mentioned programs are implemented using C++)

PART – A

1. Program to draw basic graphics primitives

```
#include <graphics.h>

#include <conio.h>


int main()

{

    int gd = DETECT, gm;

    initgraph(&gd, &gm, "");


    // Draw a Line

    line(50, 50, 200, 50);


    // Draw a Rectangle

    rectangle(50, 80, 200, 150);


    // Draw a Circle

    circle(125, 250, 50);


    // Draw an Ellipse

    ellipse(350, 250, 0, 360, 80, 40);


    // Draw an Arc

    arc(350, 100, 0, 180, 50);
```

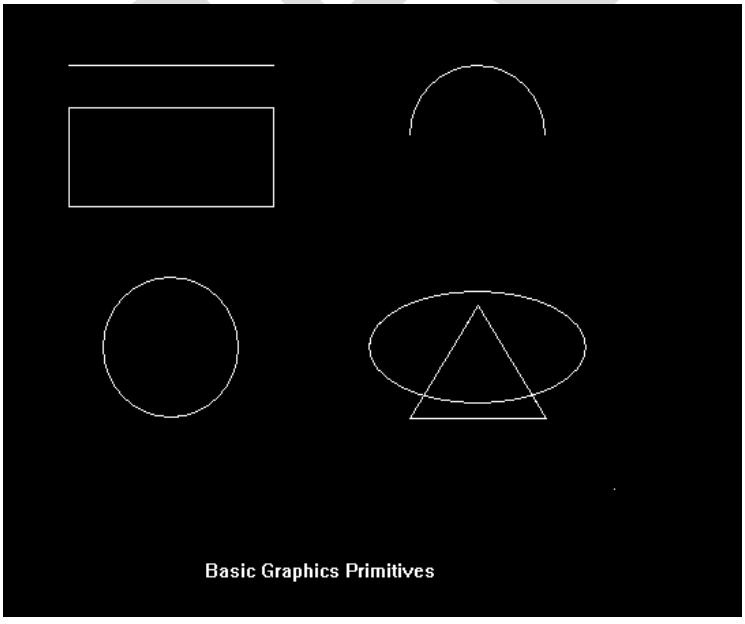
```
    // Draw a Triangle
    line(300, 300, 400, 300);
    line(300, 300, 350, 220);
    line(350, 220, 400, 300);

    // Draw a Pixel
    putpixel(450, 350, WHITE);

    // Display Text
    outtextxy(150, 400, "Basic Graphics Primitives");

    getch();
    closegraph();
    return 0;
}
```

Sample Output



2. Program to implement Digital Differential Analyzer (DDA) Line Drawing Algorithm.

```
#include <graphics.h>

#include <iostream>

#include <math.h>

#include <conio.h>


using namespace std;


int main()

{

    int gd = DETECT, gm;

    initgraph(&gd, &gm, "");


    int x1, y1, x2, y2;

    float dx, dy, steps, xInc, yInc, x, y;


    cout << "Enter x1 y1: ";

    cin >> x1 >> y1;


    cout << "Enter x2 y2: ";

    cin >> x2 >> y2;


    dx = x2 - x1;

    dy = y2 - y1;
```

```
    if (fabs(dx) > fabs(dy))
        steps = fabs(dx);
    else
        steps = fabs(dy);

    xInc = dx / steps;
    yInc = dy / steps;

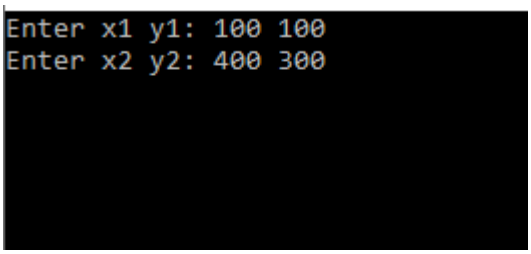
    x = x1;
    y = y1;

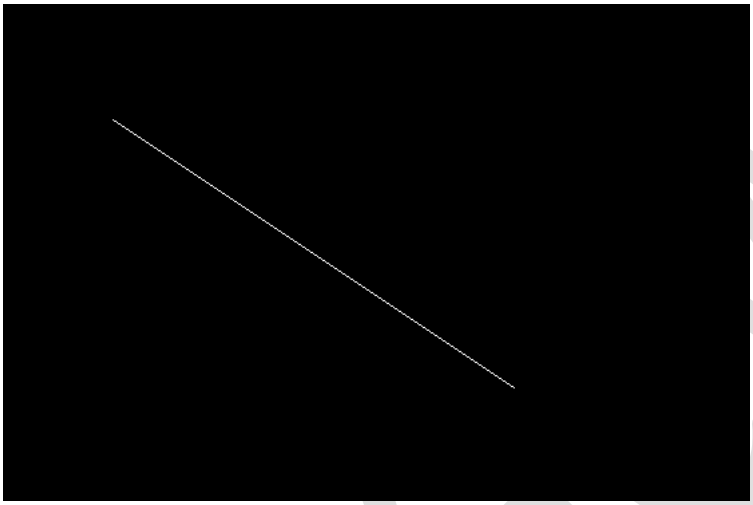
    for (int i = 0; i <= steps; i++)
    {
        putpixel(round(x), round(y), WHITE);
        x += xInc;
        y += yInc;
    }

    getch();
    closegraph();
    return 0;
}
```

Sample Output

```
Enter x1 y1: 100 100
Enter x2 y2: 400 300
```





Algorithm

1. Read the starting point `(x1, y1)` and ending point `(x2, y2)`.
2. Calculate:
   - `dx = x2 - x1`
   - `dy = y2 - y1`
3. Determine the number of steps:
   - `steps = max(|dx|, |dy|)`
4. Calculate:
   - `xInc = dx / steps`
   - `yInc = dy / steps`
5. Plot the first point.
6. Repeat `steps` times:
   - Plot the current point using `putpixel()`.
   - Increment `x` by `xInc`.
   - Increment `y` by `yInc`.

Output

A straight line is drawn between the two points entered by the user using the **DDA Line Drawing Algorithm**.

3. Program to implement Bresenham's Line Drawing Algorithm

```
#include <graphics.h>

#include <iostream>

#include <conio.h>

#include <stdlib.h>


using namespace std;


int main()

{

    int gd = DETECT, gm;

    initgraph(&gd, &gm, "");


    int x1, y1, x2, y2;

    int dx, dy, p, x, y;


    cout << "Enter x1 y1: ";

    cin >> x1 >> y1;


    cout << "Enter x2 y2: ";

    cin >> x2 >> y2;


    dx = abs(x2 - x1);

    dy = abs(y2 - y1);
```

```
    x = x1;
    y = y1;

    p = 2 * dy - dx;

    while (x <= x2)
    {
        putpixel(x, y, WHITE);

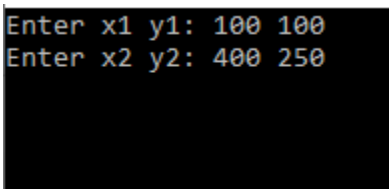
        if (p < 0)
        {
            p = p + 2 * dy;
        }
        else
        {
            y = y + 1;
            p = p + 2 * (dy - dx);
        }

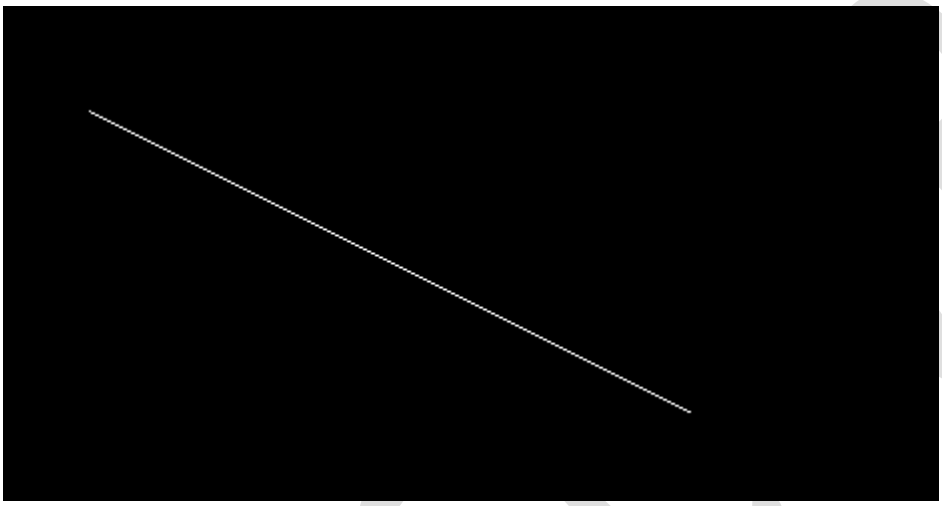
        x = x + 1;
    }

    getch();
    closegraph();
    return 0;
}
```

Sample Output

```
Enter x1 y1: 100 100
Enter x2 y2: 400 250
```





Algorithm

1. Read the starting point `(x1, y1)` and ending point `(x2, y2)`.
2. Calculate:
    - `dx = |x2 - x1|`
    - `dy = |y2 - y1|`
3. Initialize:
    - `p = 2 × dy − dx`
    - `x = x1, y = y1`
4. Plot the first point using `putpixel(x, y, WHITE)`.
5. Repeat until `x` reaches `x2`:
    - If `p < 0`, update `p = p + 2 × dy`.
    - Otherwise, increment `y` and update `p = p + 2 × (dy − dx)`.
    - Increment `x`.
    - Plot the new point.

Output

The program draws a straight line between the given coordinates using **Bresenham's Line**

4. Program to draw a circle using Bresenham's Circle Algorithm

```
#include <graphics.h>

#include <iostream>

#include <conio.h>


using namespace std;


// Function to plot 8 symmetric points

void drawCircle(int xc, int yc, int x, int y)

{

    putpixel(xc + x, yc + y, WHITE);

    putpixel(xc - x, yc + y, WHITE);

    putpixel(xc + x, yc - y, WHITE);

    putpixel(xc - x, yc - y, WHITE);


    putpixel(xc + y, yc + x, WHITE);

    putpixel(xc - y, yc + x, WHITE);

    putpixel(xc + y, yc - x, WHITE);

    putpixel(xc - y, yc - x, WHITE);

}


int main()

{

    int gd = DETECT, gm;
```

```
initgraph(&gd, &gm, "");

int xc, yc, r;
int x = 0, y, d;

cout << "Enter the center of the circle (xc yc): ";
cin >> xc >> yc;

cout << "Enter the radius: ";
cin >> r;

y = r;
d = 3 - 2 * r;

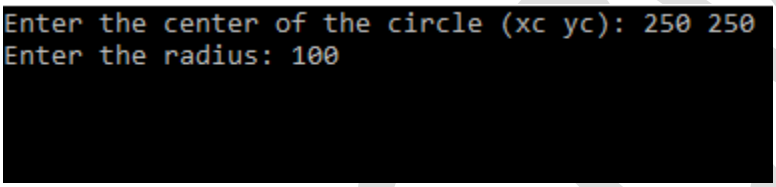
while (x <= y)
{
    drawCircle(xc, yc, x, y);

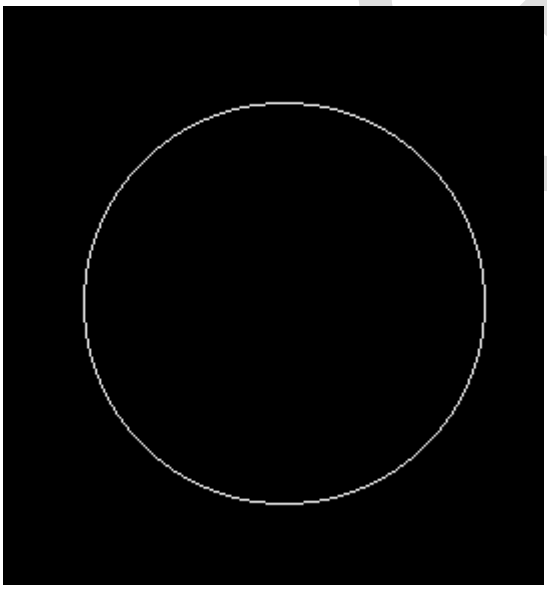
    if (d < 0)
    {
        d = d + 4 * x + 6;
    }
    else
    {
        d = d + 4 * (x - y) + 10;
        y--;
```

```
        }
        x++;
    }
    getch();
    closegraph();
    return 0;
}
```

Sample Output

```
Enter the center of the circle (xc yc): 250 250
Enter the radius: 100
```





Algorithm

1. Read the center `(xc, yc)` and radius `r`.
2. Initialize:
    - `x = 0`
    - `y = r`
    - `d = 3 - 2r`
3. While `x <= y`:
    - Plot the eight symmetric points of the circle.
    - If `d < 0`, update:

      ```
      d = d + 4x + 6
      ```

    - Otherwise:
    - ```
      d = d + 4(x - y) + 10
      y = y - 1
      ```

    - Increment `x`.
4. Continue until `x > y`.

Output

The program draws a circle with the specified center and radius using **Bresenham's Circle Drawing Algorithm**.
      ```

5. Program to perform 2D Transformations

```
#include <graphics.h>
#include <iostream>
#include <math.h>
#include <conio.h>

using namespace std;

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    // Original triangle coordinates
    int x1 = 100, y1 = 100;
    int x2 = 150, y2 = 50;
    int x3 = 200, y3 = 100;

    // Draw Original Triangle
    setcolor(WHITE);
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);

    int choice;
    cout << "2D Transformations\n";
    cout << "1. Translation\n";
    cout << "2. Scaling\n";
    cout << "3. Rotation\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch(choice)
    {
        case 1:
        {
            int tx, ty;
            cout << "Enter Translation Factors (tx ty): ";
            cin >> tx >> ty;

            setcolor(RED);
            line(x1+tx, y1+ty, x2+tx, y2+ty);
            line(x2+tx, y2+ty, x3+tx, y3+ty);
            line(x3+tx, y3+ty, x1+tx, y1+ty);
            break;
        }

        case 2:
        {
            float sx, sy;
            cout << "Enter Scaling Factors (sx sy): ";
            cin >> sx >> sy;

            setcolor(GREEN);
            line(x1*sx, y1*sy, x2*sx, y2*sy);
```

```
            line(x2*sx, y2*sy, x3*sx, y3*sy);
            line(x3*sx, y3*sy, x1*sx, y1*sy);
            break;
        }

        case 3:
        {
            float angle;
            cout << "Enter Rotation Angle (degrees): ";
            cin >> angle;

            float rad = angle * 3.14159 / 180;

            int rx1 = x1*cos(rad) - y1*sin(rad);
            int ry1 = x1*sin(rad) + y1*cos(rad);

            int rx2 = x2*cos(rad) - y2*sin(rad);
            int ry2 = x2*sin(rad) + y2*cos(rad);

            int rx3 = x3*cos(rad) - y3*sin(rad);
            int ry3 = x3*sin(rad) + y3*cos(rad);

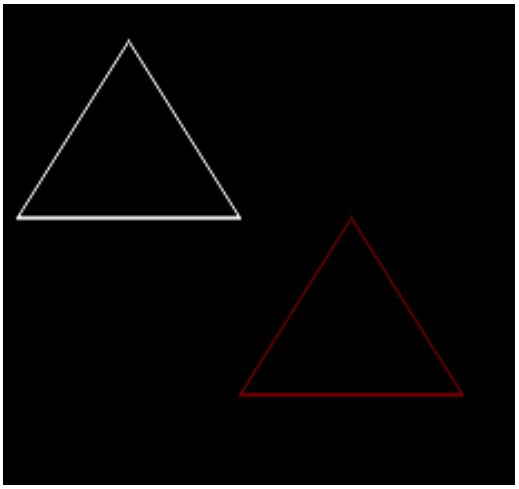
            setcolor(BLUE);
            line(rx1, ry1, rx2, ry2);
            line(rx2, ry2, rx3, ry3);
            line(rx3, ry3, rx1, ry1);
            break;
        }

        default:
            cout << "Invalid Choice!";
    }

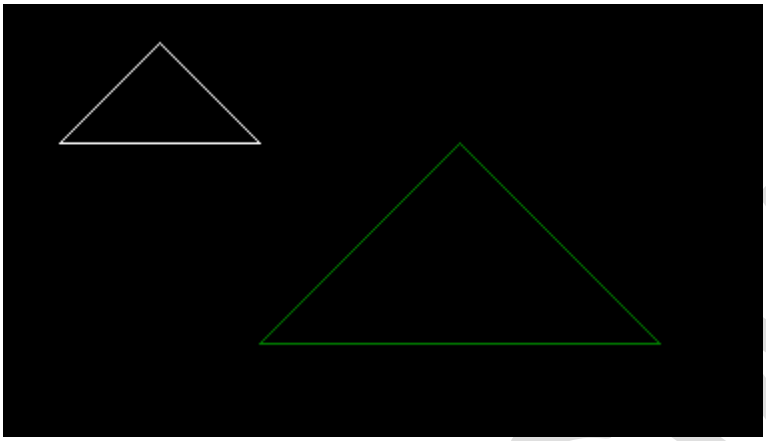
    getch();
    closegraph();
    return 0;
}
```

Sample Output

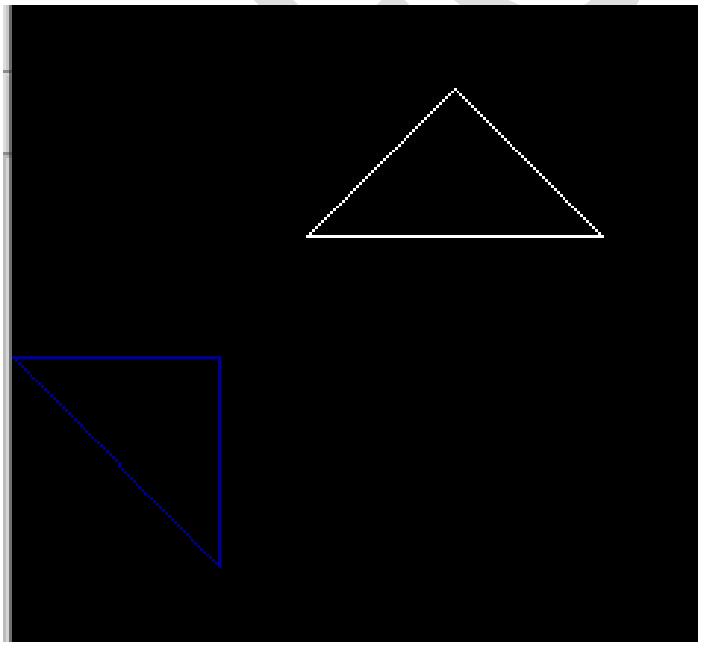
```
2D Transformations
1. Translation
2. Scaling
3. Rotation
Enter your choice: 1
Enter Translation Factors (tx ty): 100 50
```



```
2D Transformations
1. Translation
2. Scaling
3. Rotation
Enter your choice: 2
Enter Scaling Factors (sx sy): 2 2
```



```
2D Transformations
1. Translation
2. Scaling
3. Rotation
Enter your choice: 3
Enter Rotation Angle (degrees): 45
```



6. Program for Line Clipping using Cohen–Sutherland Algorithm

```
#include <graphics.h>
#include <iostream>
#include <conio.h>

using namespace std;

const int INSIDE = 0;
const int LEFT = 1;
const int RIGHT = 2;
const int BOTTOM = 4;
const int TOP = 8;

// Clipping Window
int xmin = 100, ymin = 100, xmax = 300, ymax = 300;

// Function to compute region code
int computeCode(double x, double y)
{
    int code = INSIDE;

    if (x < xmin)
        code |= LEFT;
    else if (x > xmax)
        code |= RIGHT;

    if (y < ymin)
        code |= BOTTOM;
    else if (y > ymax)
        code |= TOP;

    return code;
}

void cohenSutherlandClip(double x1, double y1, double x2, double y2)
{
    int code1 = computeCode(x1, y1);
    int code2 = computeCode(x2, y2);

    bool accept = false;

    while (true)
    {
        if ((code1 == 0) && (code2 == 0))
        {
            accept = true;
            break;
        }
        else if (code1 & code2)
        {
            break;
        }
        else
        {
            double x, y;
```

```
            int codeOut = code1 ? code1 : code2;

            if (codeOut & TOP)
            {
                x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
                y = ymax;
            }
            else if (codeOut & BOTTOM)
            {
                x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
                y = ymin;
            }
            else if (codeOut & RIGHT)
            {
                y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
                x = xmax;
            }
            else
            {
                y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
                x = xmin;
            }

            if (codeOut == code1)
            {
                x1 = x;
                y1 = y;
                code1 = computeCode(x1, y1);
            }
            else
            {
                x2 = x;
                y2 = y;
                code2 = computeCode(x2, y2);
            }
        }
    }

    if (accept)
    {
        setcolor(RED);
        line(x1, y1, x2, y2);
    }
}

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    double x1, y1, x2, y2;

    cout << "Enter x1 y1: ";
    cin >> x1 >> y1;

    cout << "Enter x2 y2: ";
    cin >> x2 >> y2;
```

```
    // Draw clipping window
    rectangle(xmin, ymin, xmax, ymax);

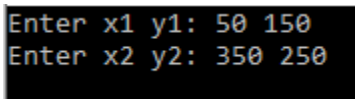
    // Draw original line
    setcolor(WHITE);
    line(x1, y1, x2, y2);

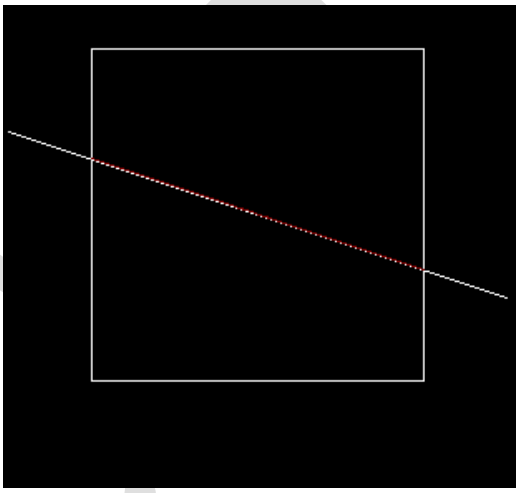
    // Clip and draw clipped line
    cohenSutherlandClip(x1, y1, x2, y2);

    getch();
    closegraph();
    return 0;
}
```

Sample Output

```
Enter x1 y1: 50 150
Enter x2 y2: 350 250
```





Output

- A **rectangle** is displayed as the clipping window.
- The **original line** is drawn in **white**.
- The **clipped line** inside the window is drawn in **red**.

Algorithm

1. Define the clipping window (`xmin`, `ymin`, `xmax`, `ymax`).
2. Compute the **region code** for both endpoints.
3. If both region codes are `0`, accept the line.
4. If the logical **AND** of both region codes is not `0`, reject the line.
5. Otherwise:
    - Find the intersection point with the clipping boundary.
    - Replace the outside endpoint with the intersection point.
    - Repeat until the line is accepted or rejected.
6. Draw the clipped line inside the clipping window.

7. Program for Polygon Clipping

```
#include <graphics.h>
#include <conio.h>
#include <iostream>

using namespace std;

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    // Clipping Window
    int xmin = 100, ymin = 100;
    int xmax = 300, ymax = 300;

    // Draw clipping window
    rectangle(xmin, ymin, xmax, ymax);

    // Original Polygon (Pentagon)
    int poly[] = {
        50,150,
        150,50,
        350,150,
        300,350,
        100,300,
        50,150
    };

    setcolor(WHITE);
    drawpoly(6, poly);

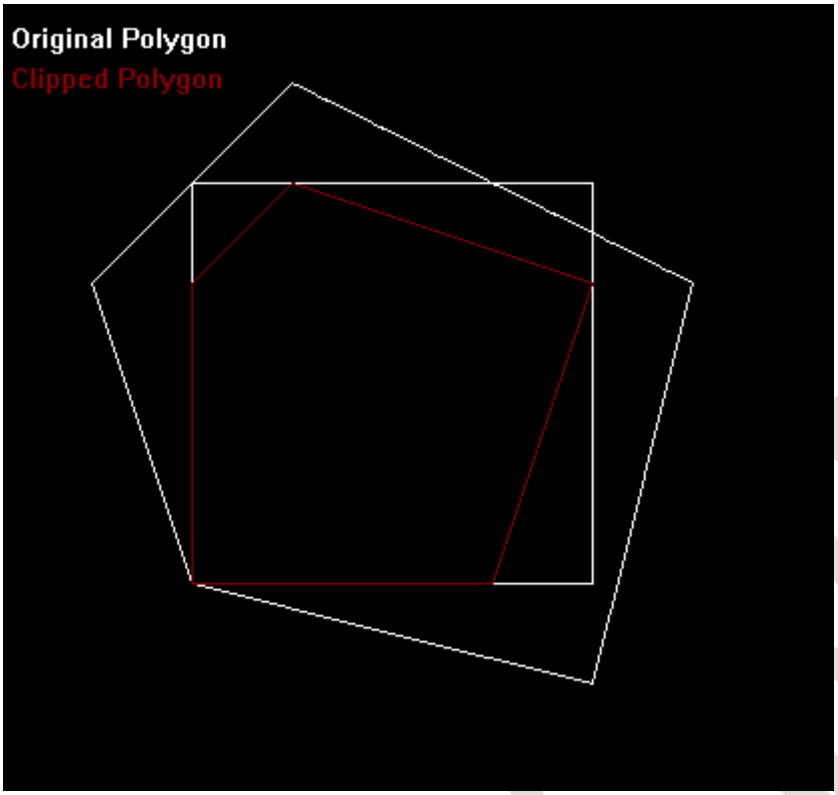
    outtextxy(10,20,"Original Polygon");

    // Clipped Polygon (Example Result)
    int clipPoly[] = {
        100,150,
        150,100,
        300,150,
        250,300,
        100,300,
        100,150
    };

    setcolor(RED);
    drawpoly(6, clipPoly);

    outtextxy(10,40,"Clipped Polygon");

    getch();
    closegraph();
    return 0;
}
```



Output

- A **rectangle** is displayed as the clipping window.
- The **original polygon** is drawn in **white**.
- The **clipped polygon** is drawn in **red**.

Algorithm (Sutherland–Hodgman Polygon Clipping)

1. Define the clipping window.
2. Draw the original polygon.
3. Clip the polygon against each boundary of the clipping window:
    - Left boundary
    - Right boundary
    - Bottom boundary
    - Top boundary
4. Calculate the intersection points wherever a polygon edge crosses a clipping boundary.
5. Draw the final clipped polygon.

8. Program to perform 3D Transformations

```
#include <graphics.h>
#include <iostream>
#include <conio.h>

using namespace std;

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    // Front face of cube
    int x1 = 100, y1 = 100;
    int x2 = 200, y2 = 100;
    int x3 = 200, y3 = 200;
    int x4 = 100, y4 = 200;

    // Depth of cube
    int d = 50;

    // Draw Original Cube
    setcolor(WHITE);

    rectangle(x1, y1, x3, y3);
    rectangle(x1+d, y1-d, x3+d, y3-d);

    line(x1, y1, x1+d, y1-d);
    line(x2, y2, x2+d, y2-d);
    line(x3, y3, x3+d, y3-d);
    line(x4, y4, x4+d, y4-d);
```

```
    // Translation values
    int tx, ty;
    cout << "Enter Translation Factors (tx ty): ";
    cin >> tx >> ty;

    // Draw Translated Cube
    setcolor(RED);

    rectangle(x1+tx, y1+ty, x3+tx, y3+ty);
    rectangle(x1+d+tx, y1-d+ty, x3+d+tx, y3-d+ty);

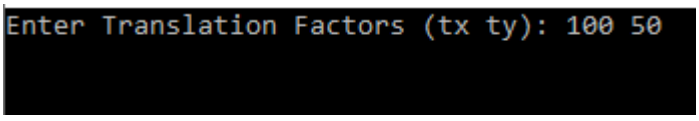
    line(x1+tx, y1+ty, x1+d+tx, y1-d+ty);
    line(x2+tx, y2+ty, x2+d+tx, y2-d+ty);
    line(x3+tx, y3+ty, x3+d+tx, y3-d+ty);
    line(x4+tx, y4+ty, x4+d+tx, y4-d+ty);

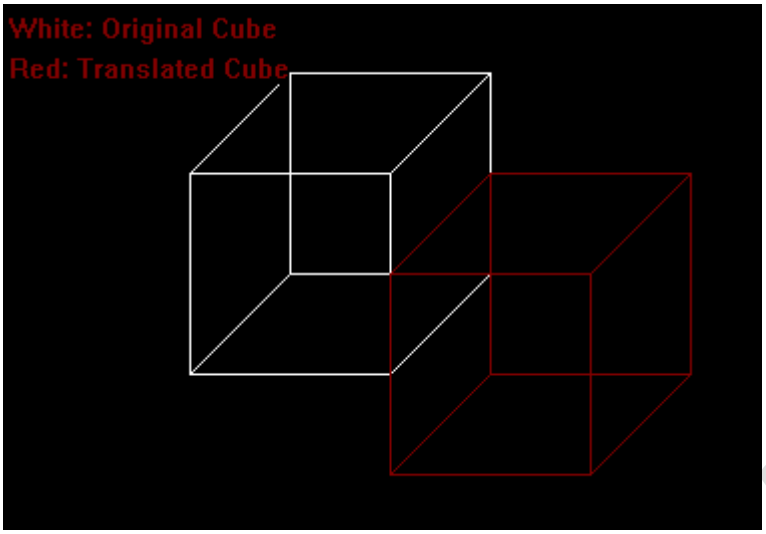
    outtextxy(10, 20, "White: Original Cube");
    outtextxy(10, 40, "Red: Translated Cube");

    getch();
    closegraph();
    return 0;
}
```

Sample Input

```
Enter Translation Factors (tx ty): 100 50
```





Output

- **White Cube** → Original 3D object.
- **Red Cube** → Translated 3D object.

Algorithm

1. Draw a simple 3D cube using two rectangles connected by lines.
2. Read the translation values `(tx, ty)`.
3. Add the translation values to all cube coordinates.
4. Draw the translated cube in a different color.